04_transfer_learning_with_resnet

October 4, 2025

[2]:
```
import os
# Check if the notebook is running on Google Colab
colab = False
if 'COLAB_GPU' in os.environ:
    colab = True
    # Running on Google Colab; install loguru
    !pip install mads_datasets mltrainer loguru
else:
    # Not running on Google Colab; you might be on a local setup
    print("Not running on Google Colab. Ensure dependencies are installed as␣
 ↪needed.")
```

```
Not running on Google Colab. Ensure dependencies are installed as needed.
```

[3]:
```
from pathlib import Path
from loguru import logger
import torch
import matplotlib.pyplot as plt
```

Let's revisit the flowers dataset from the first lesson

[4]:
```
from mads_datasets import DatasetFactoryProvider, DatasetType

flowersfactory = DatasetFactoryProvider.create_factory(DatasetType.FLOWERS)
streamers = flowersfactory.create_datastreamer(batchsize=32)
```

```
2025-10-04 19:51:44.600 | INFO     |
mads_datasets.base:download_data:121 - Folder
already exists at /Users/nickreinders/.cache/mads_datasets/flowers
```

We have just about 3000 images. To get more out of our data, we will use a technique called 'data augmentation'. When an image if flipped, or cropped, we get a different image, preventing the model to overfit on the quirks of this small dataset. We will also normalize the images to the mean and standard deviation used when training resnet; this is not strictly necessary, but should make it a bit easier for the model to work with our images.

[5]:
```
from torchvision import transforms
data_transforms = {
    'train': transforms.Compose([
```

```
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}
```

Because we want to crop the images, lets make our images during preprocessing a bit bigger, so we actually have something to crop:

[6]:
```
flowersfactory.settings
```

[6]:
```
dataset_url: https://storage.googleapis.com/download.tensorflow.org/example_imag
es/flower_photos.tgz
filename: flowers.tgz
name: flowers
unzip: True
formats: [<FileTypes.JPG: '.jpg'>]
digest: 6f87fb78e9cc9ab41eff2015b380011d
trainfrac: 0.8
img_size: (224, 224)
```

[7]:
```
flowersfactory.settings.img_size = (500, 500)
```

With this modification of the settings, we can create the dataset. We can see our images are actually 500x500 pixels now.

The transformations are just a function; we can input the img and get a transformed image out. Let try that, and visualise the result:

PS: if you dont have enough RAM on colab (eg 12GB), the cell below might crash your notebook (because it recreates the dataset); the first cell in this notebook has set the value of 'colab' to True if you are on colab, to avoid this issue.

[8]:
```
if not colab:
    datasets = flowersfactory.create_dataset()
    traindataset = datasets["train"]
    img, lab = traindataset[0]
    logger.info(f"original shape: {img.shape}")
    # original shape: torch.Size([3, 500, 500])
    transformed_img = data_transforms["train"](img)
    logger.info(f"transformed shape: {transformed_img.shape}")
    # transformed shape: torch.Size([3, 224, 224])

    fig, ax = plt.subplots(1, 2)
```

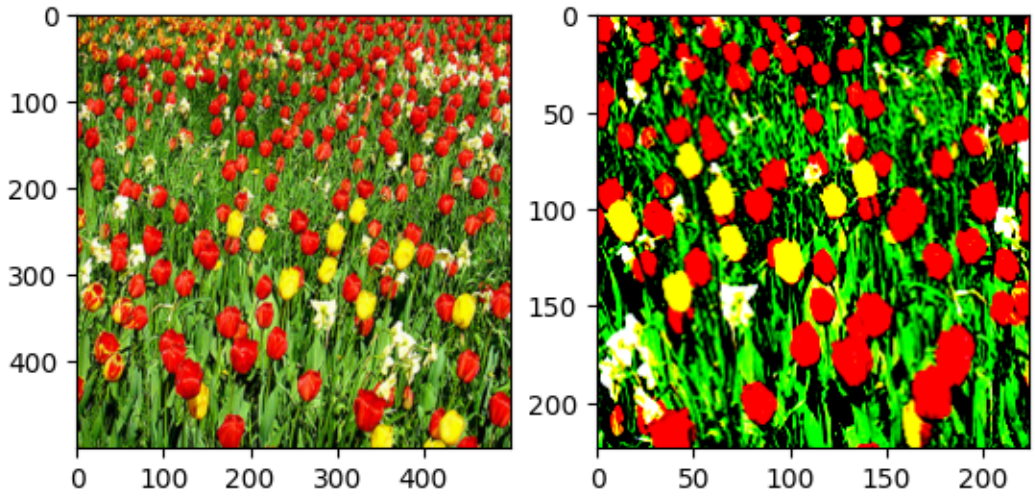
```
    ax[0].imshow(img.numpy().transpose(1, 2, 0))
    ax[1].imshow(transformed_img.numpy().transpose(1, 2, 0))
```

```
2025-10-04 19:51:51.228 | INFO     |
mads_datasets.base:download_data:121 - Folder

already exists at /Users/nickreinders/.cache/mads_datasets/flowers
2025-10-04 19:52:11.991 | INFO     |
__main__:<module>:5 - original shape:

torch.Size([3, 500, 500])
2025-10-04 19:52:12.269 | INFO     |
__main__:<module>:8 - transformed shape:

torch.Size([3, 224, 224])
Clipping input data to the valid range for imshow with RGB data ([0..1] for
floats or [0..255] for integers). Got range [-2.1147764..2.6115859].
```



Instead of using the BasePreprocessor, we will squeeze in the transformer. Lets make that:

[9]:
```
class AugmentPreprocessor():
    def __init__(self, transform):
        self.transform = transform
    def __call__(self, batch: list[tuple]) -> tuple[torch.Tensor, torch.Tensor]:
        X, y = zip(*batch)
        X = [self.transform(x) for x in X]
        return torch.stack(X), torch.stack(y)
```

Now we can create an separate preprocessor for train and validation:

[10]:
```
trainprocessor= AugmentPreprocessor(data_transforms["train"])
validprocessor = AugmentPreprocessor(data_transforms["val"])
```

And add that as the preprocessor for train and validation streamers. We do it like this because by default we can only provide a single preprocessor for both training and validation.

[11]:
```
train = streamers["train"]
valid = streamers["valid"]
train.preprocessor = trainprocessor
valid.preprocessor = validprocessor
trainstreamer = train.stream()
validstreamer = valid.stream()
```

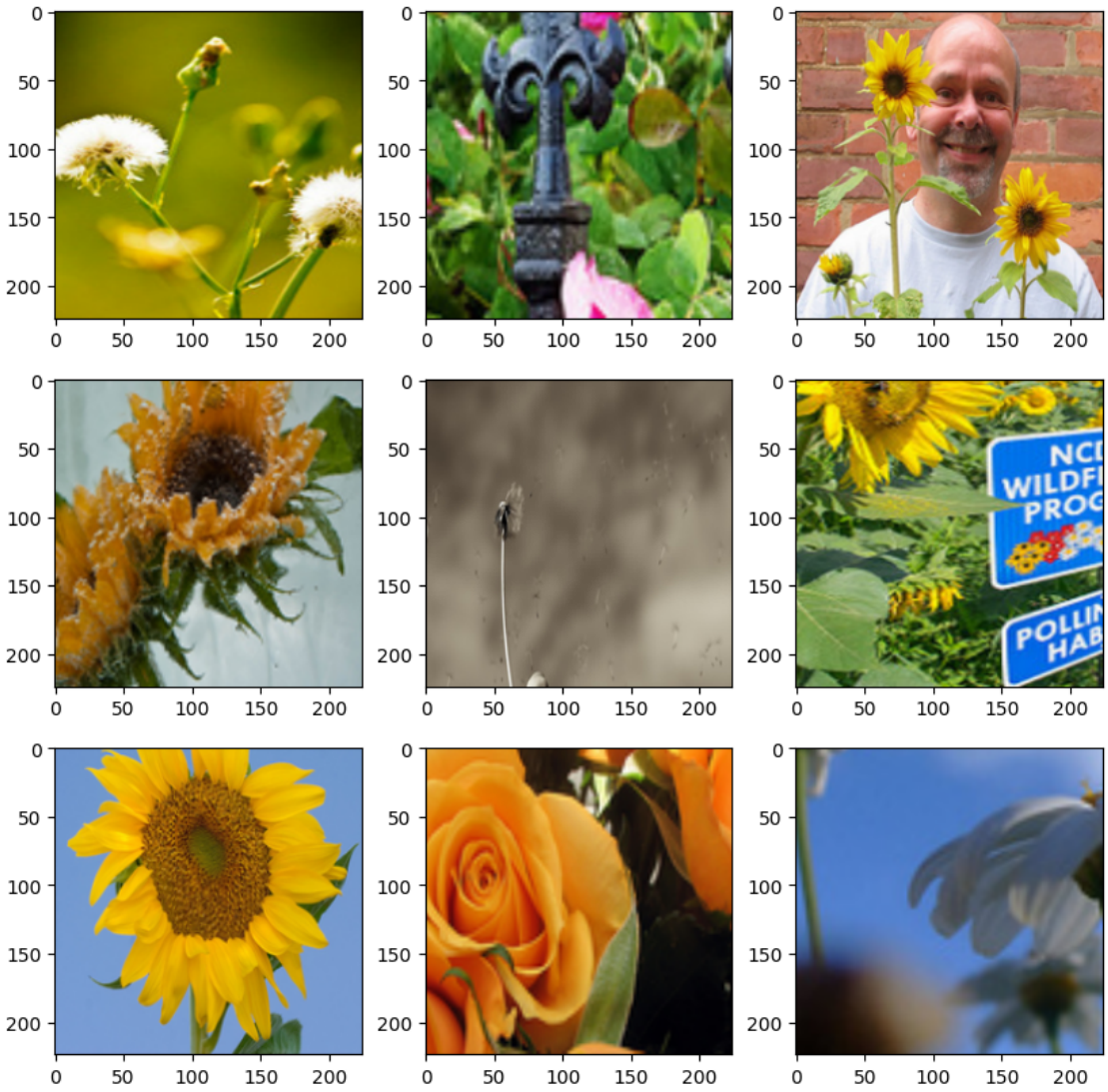
Let's confirm this works:

[12]:
```
X, y = next(trainstreamer)
X.shape, y.shape
```

[12]:
```
(torch.Size([32, 3, 224, 224]), torch.Size([32]))
```

And lets visualise a random batch of images

[13]:
```
import matplotlib.pyplot as plt
import numpy as np
img = X.permute(0, 2, 3, 1).numpy()
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
img = std * img + mean
img = np.clip(img, 0, 1)
fig, axs = plt.subplots(3, 3, figsize=(10,10))
axs = axs.ravel()
for i in range(9):
    axs[i].imshow(img[i])
```



Instead of building our own resnet, we will just download a pretrained version. This saves us many hours of training.

[14]:
```
import torchvision
from torchvision.models import resnet18, ResNet18_Weights
resnet = torchvision.models.resnet18(weights=ResNet18_Weights.DEFAULT)
```

[15]:
```
ResNet18_Weights.DEFAULT
```

[15]:
```
ResNet18_Weights.IMAGENET1K_V1
```

[16]:
```
yhat = resnet(X)
yhat.shape
```

[16]:
```
torch.Size([32, 1000])
```

However, the resnet is trained for 1000 classes. We have just 5...

We will swap the last layer and retrain the model.

First, we freeze all pretrained layers:

[17]:
```
for name, param in resnet.named_parameters():
    param.requires_grad = False
```

If you study the resnet implementation on github you can see that the last layer is named `.fc`, like this:

```
self.fc = nn.Linear(512 * block.expansion, num_classes)
```

This is a Linear layer, mapping from 512 * block.expansion to num_classes.

so we will swap that for our own. To do so we need to figure out how many features go into the .fc layer. We can retrieve the incoming amount of features for the current `.fc` with `.in_features`

[18]:
```
print(type(resnet.fc))
in_features = resnet.fc.in_features
in_features
```

```
<class 'torch.nn.modules.linear.Linear'>
```

[18]:
```
512
```

Let's swap that layer with a minimal network. Sometimes just a linear layer is enough, sometimes you want to add two layers and some dropout. Play around to see the difference!

[19]:
```
import torch.nn as nn

resnet.fc = nn.Sequential(
    nn.Linear(in_features, 5)
    #nn.Linear(in_features, 128), nn.ReLU(), nn.Dropout(0.1), nn.Linear(128, 5)
)
```

[20]:
```
yhat = resnet(X)
yhat.shape
```

[20]:
```
torch.Size([32, 5])
```

So, we have a fully trained resnet, but we added two layers at the end that transforms everything into 5 classes. These layers are random, so we need to train them for some epochs

[21]:
```
from mltrainer import metrics
accuracy = metrics.Accuracy()
```

This will take some time to train (about 4 min per epoch), you could scale down to amount of trainsteps to speed things up.

You will start with a fairly high learning rate (0.01), and if the learning stops, after patience epochs the learning rate gets halved.

[22]:
```
len(train), len(valid)
```

[22]:
```
(91, 22)
```

[23]:
```
if torch.backends.mps.is_available() and torch.backends.mps.is_built():
    device = torch.device("mps")
elif torch.cuda.is_available():
    device = torch.device("cuda")
else:
    device = "cpu"
    logger.warning("This model will take 15-20 minutes on CPU. Consider using␣
 ↪accelaration, eg with google colab (see button on top of the page)")
logger.info(f"Using {device}")
```

```
2025-10-04 19:52:18.115 | INFO     |
__main__:<module>:8 - Using mps
```

We are going to use SGD as optimizer and a stepLR as scheduler.

[24]:
```
from torch import optim
optimizer = optim.SGD
scheduler = optim.lr_scheduler.StepLR
```

To make this actually learn enough, you should increase the epochs to about 30.

[25]:
```
from mltrainer import Trainer, TrainerSettings, ReportTypes

settings = TrainerSettings(
    epochs=3,
    metrics=[accuracy],
    logdir="modellogs/flowers",
    train_steps=len(train),
    valid_steps=len(valid),
    reporttypes=[ReportTypes.TENSORBOARD],
    optimizer_kwargs= {'lr': 0.1, 'weight_decay': 1e-05, 'momentum': 0.9},
    scheduler_kwargs= {'step_size' : 10, 'gamma' : 0.1},
    earlystop_kwargs= None,
)
settings
```

[25]:
```
epochs: 3
metrics: [Accuracy]
logdir: modellogs/flowers
train_steps: 91
valid_steps: 22
reporttypes: [<ReportTypes.TENSORBOARD: 'TENSORBOARD'>]
```

```
optimizer_kwargs: {'lr': 0.1, 'weight_decay': 1e-05, 'momentum': 0.9}
scheduler_kwargs: {'step_size': 10, 'gamma': 0.1}
earlystop_kwargs: None
```

[26]:
```
trainer = Trainer(
    model=resnet,
    settings=settings,
    loss_fn=nn.CrossEntropyLoss(),
    optimizer=optimizer,
    traindataloader=trainstreamer,
    validdataloader=validstreamer,
    scheduler=scheduler,
    device=device,
    )
```

```
2025-10-04 19:52:18.164 | INFO     |
mltrainer.trainer:dir_add_timestamp:24 - Logging

to modellogs/flowers/20251004-195218
```

[27]:
```
# note: this will be very slow without acceleration!
trainer.loop()
```

```
100%|      | 91/91 [00:20<00:00,  4.46it/s]
2025-10-04 19:52:42.092 | INFO     |
mltrainer.trainer:report:209 - Epoch 0 train

4.0458 test 1.6661 metric ['0.8736']
100%|      | 91/91 [00:11<00:00,  8.14it/s]
2025-10-04 19:52:55.201 | INFO     |
mltrainer.trainer:report:209 - Epoch 1 train

3.5158 test 4.6391 metric ['0.7756']
100%|      | 91/91 [00:09<00:00,  9.33it/s]
2025-10-04 19:53:06.834 | INFO     |
mltrainer.trainer:report:209 - Epoch 2 train

3.4645 test 2.1711 metric ['0.8679']
100%|      | 3/3 [00:48<00:00, 16.17s/it]
```

[ ]:
```

```